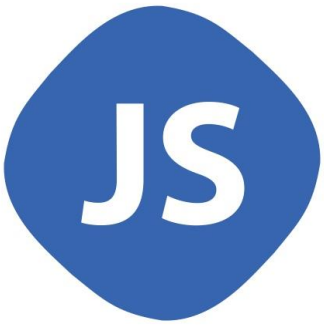


Объект Event

- Объект события (Event)
- Захват (погружение) и всплытие событий
- Всплытие событий
- Захват (погружение) событий
- Действия по умолчанию
- Делегирование событий

Объект события (Event)

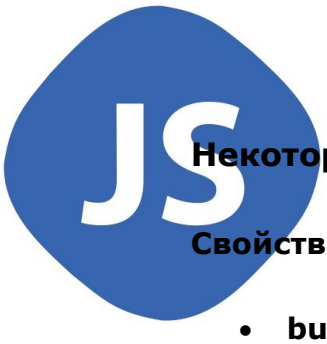
Иногда внутри функции обработчика событий разработчик использует параметр, заданный таким именем, как **event** или **evt** или просто **e**. Называется этот параметр – объект события **Event**. Объект **автоматически передаётся обработчикам событий** для предоставления дополнительных функций и информации.

> ***Примечание***. Для объекта события вместо **event**/**evt**/**e** можно использовать любое другое имя, которое затем можно использовать для ссылки в функции обработчика событий. Но, предложенные имена чаще всего используются разработчиками. Всегда придерживайтесь стандарта.

Объект Event описывает событие, произошедшее на странице. Одной из причин возникновения событий являются действия пользователя, такие как клики мышкой **MouseEvent** или ввод с клавиатуры **KeyboardEvent**.

Когда происходит событие, браузер создаёт **объект события**, записывает в него детали и передаёт его в качестве аргумента функции-обработчику.

Существует множество различных событий с разным набором информации. **Каждое событие обладает набором определенных свойств и методов**.



Некоторые свойства и методы объекта Event

Свойства:

- **bubbles** — является ли данное событие всплывающим.
- **cancelable** — является ли событие отменяемым.
- **currentTarget** — указывает на элемент, на котором установлен обработчик события.
- **defaultPrevented** — отменено ли поведение события по умолчанию.
- **eventPhase** — указывает на фазу срабатывания события.
- **isTrusted** — указывает на происхождение события, будет в значении **true**, если событие инициировано действиями пользователя. **false** — если событие инициировано из кода с помощью **dispatchEvent**().
- **target** — ссылка на объект, которым было инициировано событие. Например, если событие произошло на поле ввода, мы получим ссылку на этот DOM элемент.
- **timeStamp** — время возникновения события в миллисекундах.
- **type** — тип события.

Методы:

- **composedPath**() — вернёт массив элементов, на которых сработает событие.
- **preventDefault**() — предотвращает дефолтное поведение события. Если вызвать этот метод на событии клика по ссылке, то переход по ссылке не произойдёт, но событие продолжит всплытие.
- **stopPropagation**() — предотвращает всплытие события.
- **stopImmediatePropagation**() — делает то же самое, что и stopPropagation, но, в том числе, предотвращает вызов обработчиков события, которые были установлены на этом же элементе.

Одним из важнейших свойств является свойство **target** (event.target).

> ***Важно.*** Свойство объекта события **target** всегда является **ссылкой на элемент, на которым только что произошло событие**.

Именно вокруг свойства **target** будет выстроен программный код многих решаемых задач языка программирования JS.

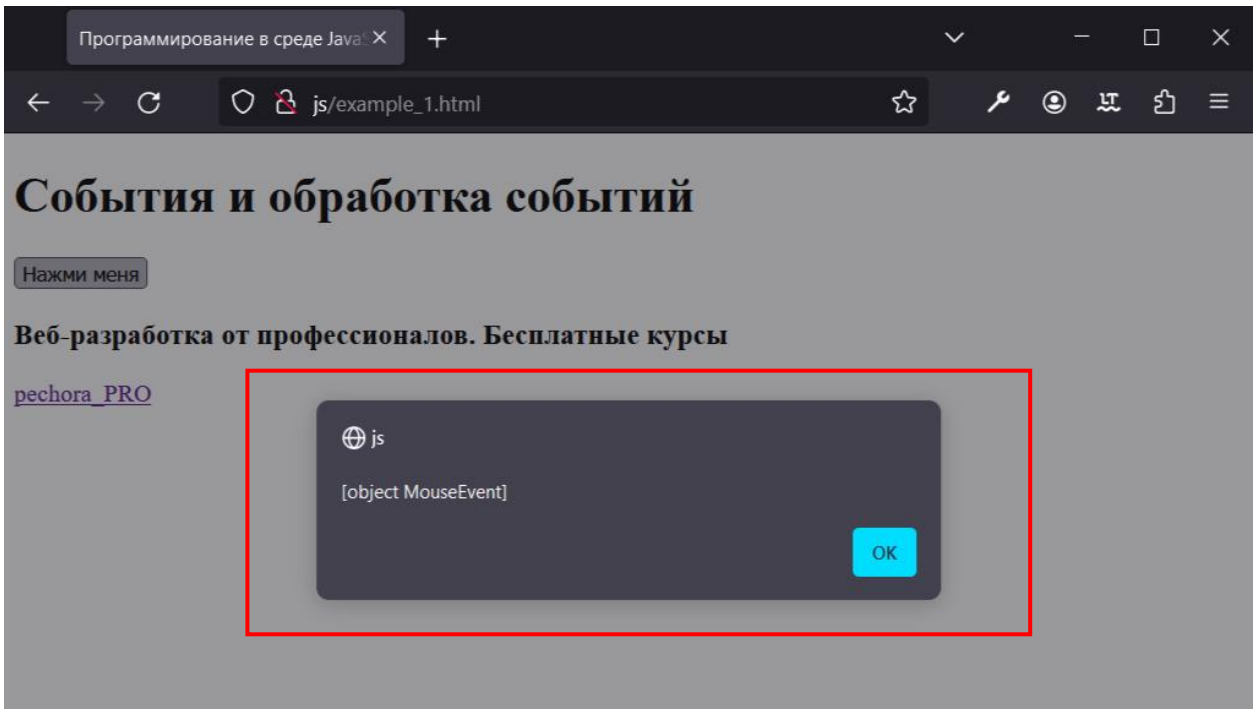
Итак, событие **Event** в демонстрационном примере **example_1**.

example_1.

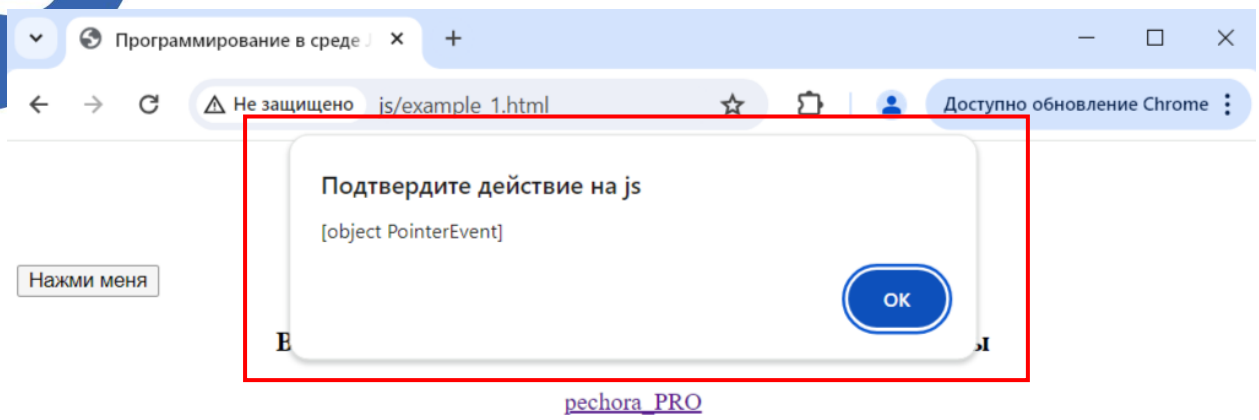
```
<input type="button" onclick="testEvent()" value="Нажми меня">
<script>
    function testEvent() {
        // вывод события
        alert(event);
        // подробно об объекте события
        console.log(event);
    }
</script>
```

Обратите внимание на вывод **alert(event)** для разных браузеров.

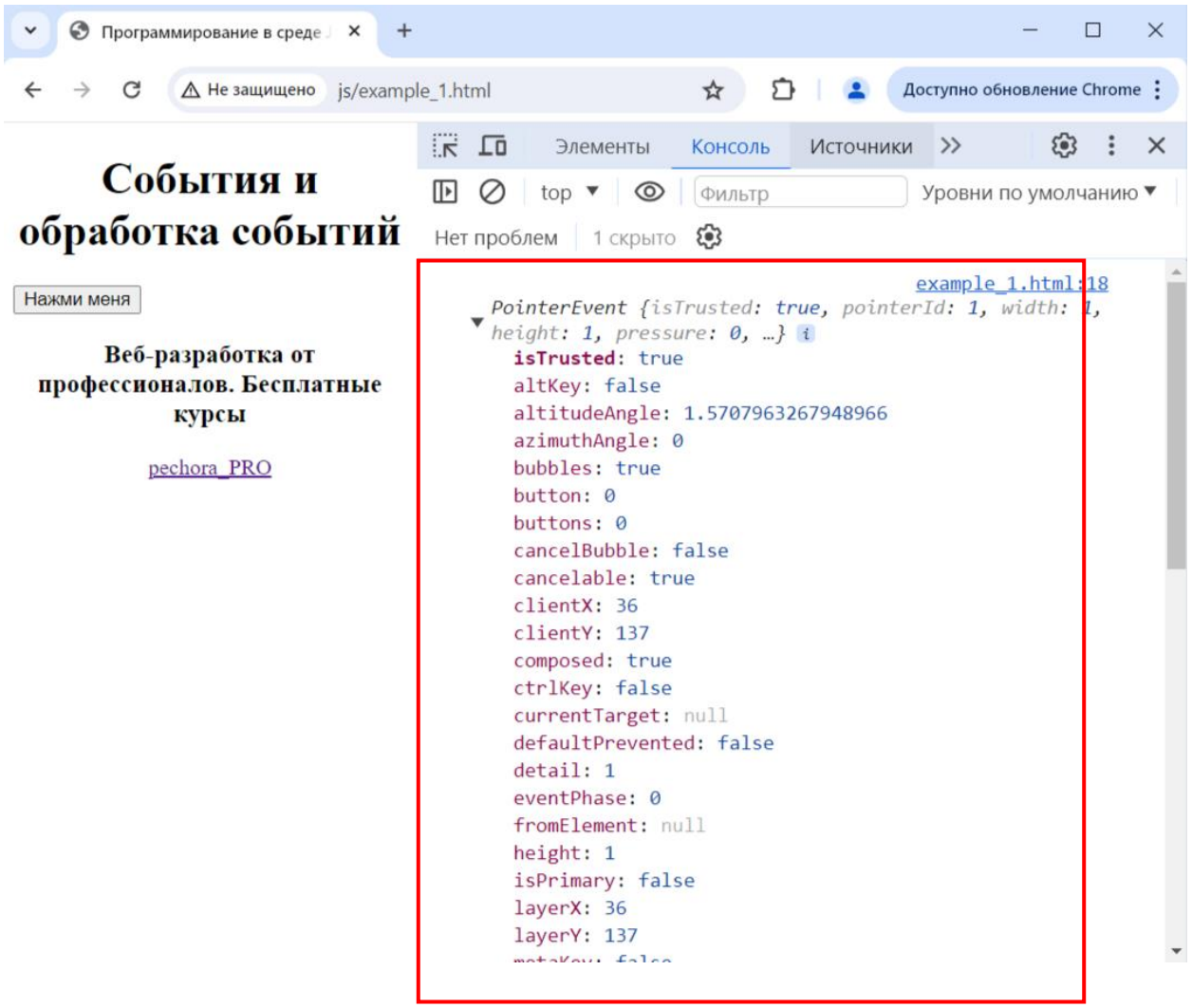
Браузер **Mozilla Firefox**:



Браузер **Google Chrome**:



Полный список свойств объекта **Event** можно увидет в консоле.





В демонстрационном примере **example_2** рассмотрим **координаты курсора относительно окна браузера** в момент клика по HTML-элементам страницы.

example_2.

```
<input class="testEvent" type="button" value="Узнай 10 принципов успеха!">
<p><a class="testEvent" href="#">Нажми и забирай приз!</a></p>
<p class="testEvent">Просто сделай это...</p>
<div class="testEvent">Жми. Невозможного не существует!</div>
<p><button class="testEvent">Хотите, я вам на рояле поиграю?</button></p>
<script>
    // коллекция элементов .testEvent
    elems = document.getElementsByClassName("testEvent");
    // назначим событие каждому элементу коллекции
    for(el of elems) {
        el.onclick = function() {
            // вывод тип события
            alert("Тип события: " + event.type + " на элементе: "
            + event.target);
            // вывод названия и координат элемента коллекции
            console.log("Элемент: " + event.target.tagName);
            console.log("Координаты: " + event.clientX + ":" +
            event.clientY);
            console.log("----------");
        }
    };
</script>
```



В следующем примере разберем ряд других, наиболее востребованных разработчиками свойств.

example_3.

```
<div id="testEvent">
    <input id="btn" class="button" type="button" value="Нажми меня">
    <p><a id="link" class="link" href="#">Лучше меня</a></p>
    <p id="par" class="paragraph">Или меня</p>
    <div id="dv" class="div">С тебя кликбейт, дружище...</div>
    <p><button id="but" class="button">А хотите, я вам на рояле поиграю?</button></p>
</div>
<!-- контейнер для вывода -->
<div id="con"></div>
<script>
    // коллекция элементов блока #testEvent
    elems = document.getElementById("testEvent").children;
    // назначим событие каждому элементу коллекции
    for(el of elems) {
        el.onclick = function() {
            // формирование данных об элементе коллекции
            let info = `
                Тип события: ${event.type} <br>
                Тег: ${event.target.tagName} <br>
                id: ${event.target.id} <br>
                className: ${event.target.className} <br>
```

```
                innerHTML: ${event.target.innerHTML} <br>
                nodeType: ${event.target.nodeType} <br>
            `;
            // вывод данных
            con.innerHTML = info;
        }
    };
</script>
```

С самим объектом события все более-менее понятно. Гораздо труднее с его поведением. Разбору поведения событий посвятим следующие разделы урока.

Захват (погружение) и всплытие событий

Когда событие инициируется элементом, который имеет родительские элементы, современные браузеры выполняют **три разные фазы**:

- **фазу захвата (погружение)** (capturing phase)
- **фазу выбора цели** (target phase)
- **фазу всплытия** (bubbling stage)

На стадии захвата (capture) происходит следующее:

Браузер проверяет, имеет ли самый внешний элемент (<html>) обработчик события onclick, зарегистрированный на нем на этапе захвата и запускает его, если это так.

Затем он переходит к следующему элементу внутри <html> и выполняет то же самое, затем следующее и так далее, пока не достигнет элемента, на который на самом деле нажали.

На стадии выбора (target) цели:

Событие достигло целевого(исходного) элемента.

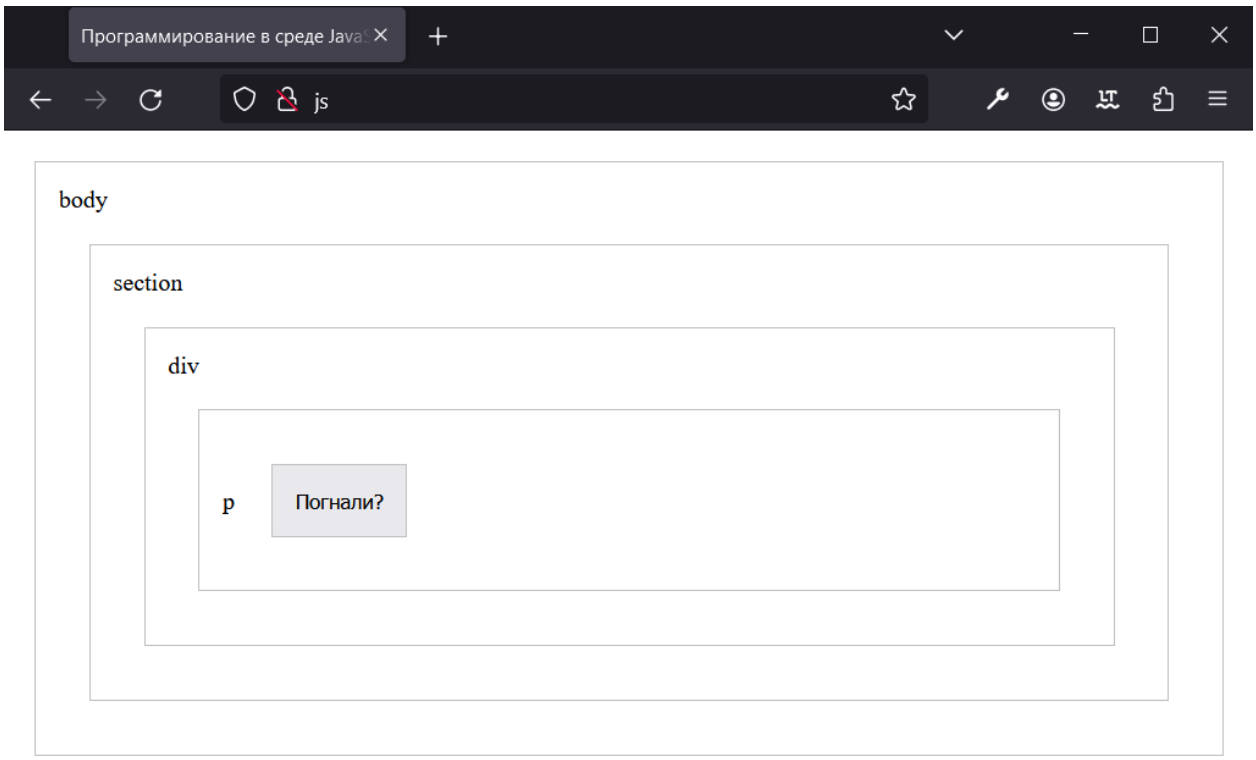
На стадии всплытия (bubbling) происходит полная противоположность:

Браузер проверяет, имеет ли элемент, который был фактически нажат, обработчик события onclick, зарегистрированный на нем в фазе всплытия, и запускает его, если это так.

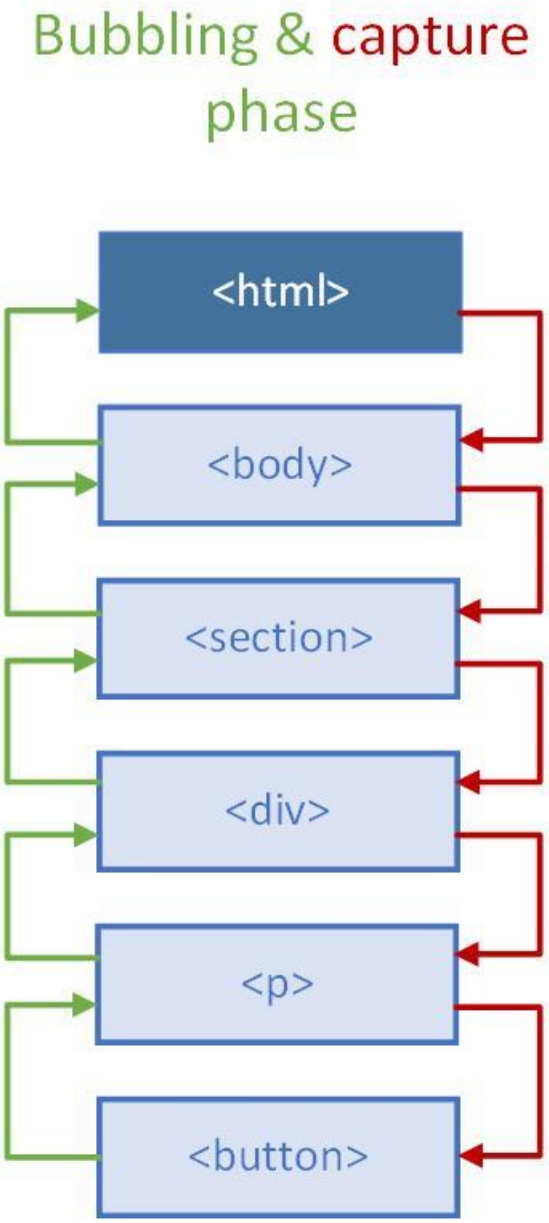
Затем он переходит к следующему непосредственному родительскому элементу и выполняет то же самое, затем следующее и так далее, пока не достигнет элемента <html>.

> ***Важно.*** В современных браузерах по умолчанию **все обработчики событий регистрируются в фазе всплытия**.

В течении текущего урока мы будем работать со следующей **структурой** вложенных элементов:



Структурно отобразить происходящие фазы можно следующей схемой.



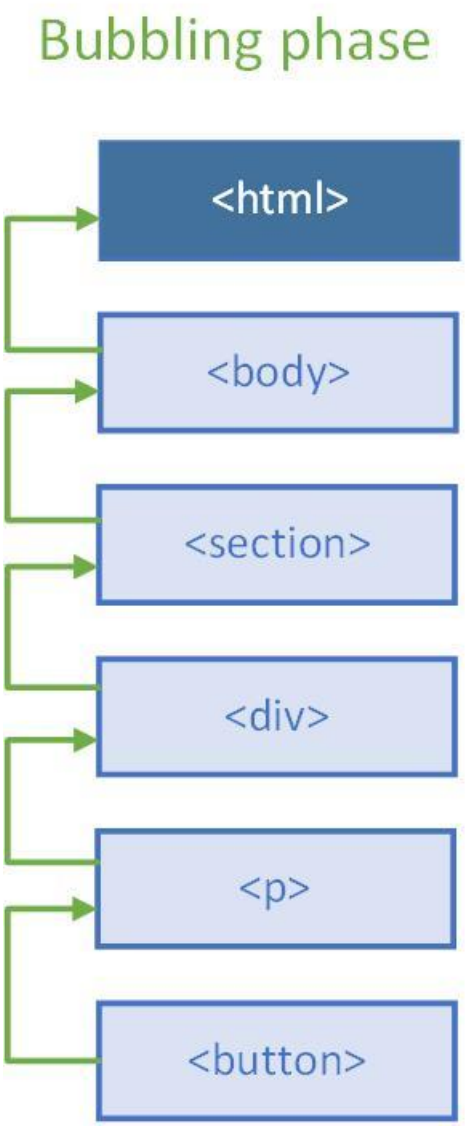
Всплытие событий

Принцип всплытия очень простой. Когда на элементе происходит событие, обработчики сначала срабатывают на нём, потом на его родителе, затем выше и так далее, **вверх по цепочке предков**.

> ***Важно***. Когда на **элементе происходит событие**, обработчик этого события сначала срабатывает на самом элементе, потом на его родителе,

затем выше и так далее, вверх по цепочке предков.

Процесс **фазы всплытия** представлен в следующей схеме.



Клик по элементу <button> вызовет обработчик **onclick**:

- сначала на самом <button>
- потом на внешнем <p>
- затем на внешнем
- затем на внешнем <section>
- затем на внешнем <body>
- и так далее вверх по цепочке до самого <html>.



Этот процесс называется **всплытием**, потому что события **всплывают** от внутреннего элемента вверх через родителей подобно тому, как всплывает пузырёк воздуха в воде (ну вот такое сравнение).

> ***Важно***. Почти все события, за редким исключением, имеют фазу всплытия.

В следующих демонстрационных примерах поработаем с фазой **всплытия**.

В демонстрационном примере **example_4** переменной **num** при клике по элементу <button> присвоили значение 1. На верхнем уровне всплытия эта переменная имеет значение 5.

example_4.

```
<html lang="ru" onclick="alert(num)"> <!-- 5 -->
    <!-- ... -->
    <body class="elem" onclick="num++">body
        <section class="elem" onclick="num++">section
            <div class="elem" onclick="num++">div
                <p class="elem" onclick="num++">p
                    <!-- старт событиям onclick -->
                    <button class="elem"
                    onclick="num=1">Погнали?</button>
                </p>
            </div>
        </section>
    </body>
```

В примере **example_5** отлично показан эффект всплытия. Протестируйте пример, далее будут нюансы.

example_5.

```
<html lang="ru" onclick="alert('Тег html')">
    <!-- ... -->
    <body class="elem" onclick="alert('Тег body')">body
        <section class="elem" onclick="alert('Тег
        section')">section
            <div class="elem" onclick="alert('Тег div')">div
                <p class="elem" onclick="alert('Тег p')">p
                    <!-- старт событиям onclick -->
                    <button class="elem"
                    onclick="outMessage()">Погнали!</button>
                </p>
            </div>
        </section>
        <script>
            function outMessage() {
                alert ("Ой... Здравствуйте!");
            }
        </script>
    </body>
```

Допустим, я не хочу навешивать обработчики как HTML-атрибут элемента (и на это есть веские основания). Используем метод **addEventListener**().

Обратите внимание, в демонстрационном примере **example_6** представлен программный код с часто допускаемой **ошибкой объявления переменных**. Протестируйте пример.

example_6.

```
<body class="elem">body
```

```
<section class="elem">section
    <div class="elem">div
        <p class="elem">p
            <!-- старт событиям onclick -->
            <button class="elem">Погнали :((</button>
        </p>
    </div>
</section>
<script>
    // пример неправильного кода
    for(elem of document.querySelectorAll('.elem')){
        // вывод имени тега узла на каждой итерации
        console.log(elem.tagName);
        // каждому элементу коллекции навесим обработчик
        elem.addEventListener ("click", () =>
        alert(elem.tagName));
    }
    // обратите внимание на содержание elem в последней
    итерации (BUTTON)
    console.log("----------");
    console.log(elem);
</script>
```

А теперь пример того, как писать правильно.

example_7.

```
<body class="elem">body
    <section class="elem">section
        <div class="elem">div
            <p class="elem">p
                <!-- старт событиям onclick -->
                <button class="elem">Погнали ;)</button>
            </p>
        </div>
    </section>
    <script>
        // обратите внимание на let в объявлении elem
        for(let elem of document.querySelectorAll('.elem')){
            // вывод имени тега узла на каждой итерации
            console.log(elem.tagName);
            // каждому элементу коллекции навесим обработчик
            // в качестве обработчика - объект с методом
            handleEvent
            elem.addEventListener ("click",
            {handleEvent:userClick, el:elem});
        }
        function userClick(){
            alert(this.el.tagName)
        }
    </script>
```

Целевой элемент. Свойство event.target

Всегда можно узнать, на каком конкретно элементе произошло событие.

> ***Важно***. Самый глубокий элемент, который вызывает событие, называется **целевым элементом**, и он доступен через свойство **event.target**.

Отличия **event.target** от **this** (event.currentTarget):

- **event.target** – это целевой элемент, на **котором произошло событие**, в процессе всплытия он неизменен.
- **this** – это текущий элемент, до которого дошло всплытие, на нём сейчас **выполняется обработчик**.

Например, в следующем примере обработчик:

```
document.body.onclick = function () { ... }
```

поймает все клики внутри вложенной структуры элементов. Где бы ни был клик внутри – он **всплывёт** до элемента <body>, на котором сработает обработчик.

При этом внутри обработчика:

- **this** (event.currentTarget) всегда будет элемент <body>, так как обработчик сработал на нем;
- **event.target** будет содержать ссылку на целевой элемент внутри элемента <body>, на котором произошёл клик.

example_8.

```
<body class="elem">body
    <section class="elem">section
        <div class="elem">div
            <p class="elem">p
                <!-- старт событиям onclick -->
                <button class="elem">Погнали...</button>
            </p>
```

```
    </div>
</section>
<script>
    // регистрируем обработчик на click по body
    document.body.onclick = function handlerClick(){
        alert(event.target.tagName); // целевой элемент
        alert(this.tagName); // всегда body
        console.log(event.currentTarget.tagName); // всегда body
    }
</script>
```

Прекращение всплытия. Метод stopPropagation()

Всплытие идёт с **целевого** элемента прямо наверх. По умолчанию событие будет всплывать до элемента <html>, вызывая все обработчики на своём пути.

Но любой промежуточный обработчик может решить, что событие полностью обработано, и всплытие необходимо остановить

Для этого нужно вызвать метод **stopPropagation**() (event.stopPropagation()).

В демонстрационном примере **example_9** протестируйте метод **stopPropagation**() на разных HTML-элементах страницы браузера.

example_9.

```
<body class="elem" onclick="alert(this.tagName)">body
    <section class="elem" onclick="alert(this.tagName);
    event.stopPropagation();">section
        <div class="elem" onclick="alert(this.tagName)">div
            <p class="elem" onclick="alert(this.tagName)">p
                <!-- старт событиям onclick -->
```

```
            <button class="elem"
            onclick="outMessage()">Погнали (чуть-
            чуть)</button>
        </p>
    </div>
</section>
<script>
    function outMessage() {
        alert ("Ой... Здравствуйте!");
        // event.stopPropagation();
    }
</script>
```

Важно. Не имея веской причины, **не стоит прерывать** фазу всплытия.

Захват (погружение) событий

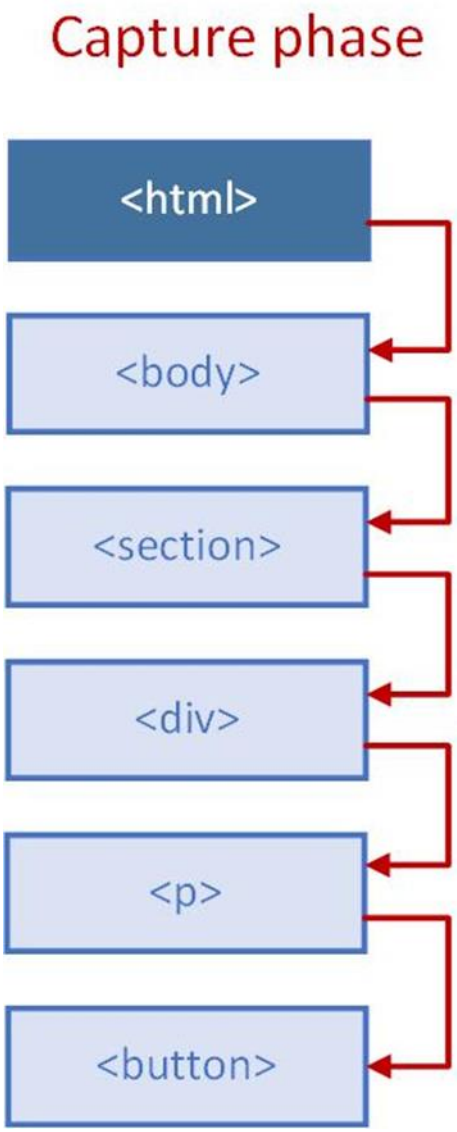
Существует ещё одна фаза из жизненного цикла события – **захват** (или погружение).

Стандарт **DOM Events** описывает три фазы прохода события:

- Фаза **погружения** – событие сначала идёт сверху вниз.
- Фаза **цели** – событие достигло целевого(исходного) элемента.
- Фаза **всплытия** – событие начинает всплывать.

Фаза захвата (погружения) редко используется в **реальном коде**, однако тоже может быть полезной. Рассмотрим фазу на уже известной нам структуре элементов.

Процесс **фазы погружения** представлен в следующей схеме.



То есть при клике на элементе <button> **событие** путешествует по цепочке родителей:

- сначала **вниз** к элементу (погружается),
- затем достигает **целевой** элемент (фаза цели),
- потом идёт **наверх** (всплытие), вызывая по пути обработчики.

Чаще всего в веб-разработке будет использоваться фаза **всплытия**, другие стадии, как правило, не используются и проходят незаметно для пользователя.

Важно. В современной разработке стадия погружения используется очень

Чаще, обычно события обрабатываются **во время всплытия**.

Обработчики, добавленные через:

- **HTML-атрибуты**
- **свойство узла**
- **addEventListener(event, handler)** с двумя аргументами,

ничего не знают о фазе погружения, а работают только на втрой и третьей фазах.

Чтобы поймать событие на стадии погружения, нужно использовать третий аргумент **capture** вот так:

```
elem.addEventListener(..., {capture: true})
```

или просто **true**, как сокращение для {capture: true}

```
elem.addEventListener(..., true)
```

Существуют два варианта значений опции **capture**:

- Если аргумент **false** (по умолчанию), то событие будет поймано при всплытии.
- Если аргумент **true**, то событие будет перехвачено при погружении.

Структура вложенных элементов та же, программный код не изменился (ну, почти), но теперь выполняем **погружение**.

example_10.

```
<body class="elem">body

    <section class="elem">section

        <div class="elem">div

            <p class="elem">p

                <!-- старт событиям onclick -->

                <button class="elem">Погнали</button>
```

```
            </p>
        </div>
    </section>
    <script>
        // переберем коллекцию элементов .elem
        for(let elem of document.querySelectorAll('.elem')){
            // название тега текущего элемента
            console.log(elem.tagName);
            // вывод названия тега в диалог
            elem.addEventListener ("click", () =>
            alert(elem.tagName), {capture: true});
        }
    </script>
```

Таким образом, объединив все вышесказанное можно получить следующую картину **погружений** и **всплытий**. При анализе кода будьте внимательны, события будут сначала погружаться и уже потом всплывать.

example_11.

```
<body class="elem">body
    <section class="elem">section
        <div class="elem">div
            <p class="elem">p
                <!-- старт событиям onclick -->
                <button class="elem">Погнали</button>
            </p>
        </div>
    </section>
    <script>
```

```
// переберем коллекцию элементов .elem
for(let elem of document.querySelectorAll('.elem')){
    // название тега текущего элемента
    console.log(elem.tagName);
    // клик реагирующий на погружение
    elem.addEventListener ("click", () => alert('Погружение
    (Capture): ' + elem.tagName), {capture: true});
    // клик реагирующий на всплытие
    elem.addEventListener ("click", () => alert('Всплытие
    (Bubbling): ' + elem.tagName), {capture: false});
}
</script>
```

Действия по умолчанию

Для многих браузерных событий есть **действия по умолчанию**, такие как переход по ссылке, ввод текста, выделение и т.п.

Метод **preventDefault**() интерфейса **Event** сообщает, что если событие не обрабатывается явно, его действие по умолчанию **не должно выполняться так, как обычно**. Событие продолжает распространяться как обычно, до тех пор, пока один из его обработчиков не вызывает методы **stopPropagation** () или **stopImmediatePropagation** (), любой из которых сразу же **прекращает распространение**.

Важно. Вызов **preventDefault**() является возможностью для обработчика события сообщить в сгенерировавший событие код, что эти действия надо **отменить**.



Для пользовательских событий браузерных действий, не предусмотрено, но код, который генерирует такое событие, может предусматривать какие-то свои действия после события (пользовательские события – следующий урок).

Далее – действие по умолчанию и **ссылка**.

example_12.

```
<a id="link" href="target.html">Ссылка на файл target.html</a>
<script>
    // навесим на ссылку событие и его обработчик
    link.onclick = () => {
        // отменим действие по умолчанию (переход по ссылке)
        event.preventDefault();
        alert("Целевой файл временно недоступен!");
    }
</script>
```

Действие по умолчанию и **флажок**.

example_13.

```
<form>
    Сделайте одолжение, кликните по флажку:
    <input type="checkbox" id="chk" />
</form>
<script>
    function stopClick(e) {
        // отменим действие по умолчанию (изменение состояния
        флажка)
        e.preventDefault();
```

```
        alert("Нечего тут... Дома у себя будете кликать!");
    }
    // навесим на флажок событие и его обработчик
    document
        .getElementById("chk")
        .addEventListener("click", stopClick, false);
</script>
```

Действие по умолчанию и **форма**.

example_14.

```
<p>Пожалуйста, введите логин, используя буквы нижнего регистра латиницы:</p>
<form>
    <input type="text" id="login">
</form>
<script>
    // навесим на текстовое поле событие  и его обработчик
    login.addEventListener("keypress", checkInput);
    function checkInput(e) {
        // получим код введенного символа
        var chrCode = e.charCode;
        if (chrCode < 97 || chrCode > 122) {
            // отменим действие по умолчанию (ввод символа)
            e.preventDefault();
            // выведем предупреждение пользователю
            alert("Используйте буквы нижнего регистра латиницы");
```

```
            // служебное сообщение
            console.log("Недопустимый символ: " + chrCode);
        }
    }
</script>
```

Примечание. В одной из **самостоятельных работ** продолжим тему ввода данных в поля формы и проверку введенных символов с помощью регулярных выражений.

Делегирование событий

Всплытие также позволяет использовать **делегирование событий**. Если у какого-либо родительского элемента есть множество дочерних элементов, и вы хотите, чтобы определённый код выполнялся при клике (событии) на каждом из дочерних элементов, можно **установить обработчик событий на родительском элементе** и события, происходящие на дочерних элементах, будут всплывать до их родителя.

Важно. При этом **не нужно** устанавливать обработчик событий на каждом дочернем элементе.

Хорошим примером является группа элементов **списка**. Если вы хотите, чтобы каждый из элементов, например, отображал сообщение при нажатии, вы можете установить обработчик событий **click** для родительского элемента (ul, ol), и он будет всплывать в каждом элементе списка.

example_15.

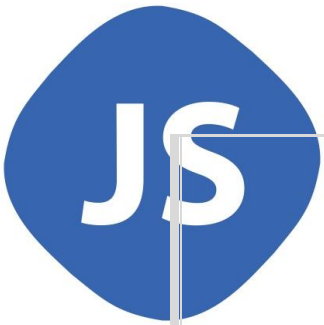
```
<body>body
```

```
<section>section
    <div>div
        <p>p
            <button>Погнали</button>
        </p>
    </div>
</section>
<script>
    // назначим обработчик события click для body
    document.body.onclick = function userClick(){
        // отследим, кто является инициатором события
        alert("Выполнен клик по элементу: " +
        event.target.tagName);
        // всегда body
        console.log(this.tagName);
    }
</script>
```

А еще хорошим примером делегирования событий являются **таблицы** с их хорошо структурированной разметкой. Решим проблему управления строками таблицы, которую я озвучил в предыдущем уроке.

example_16.

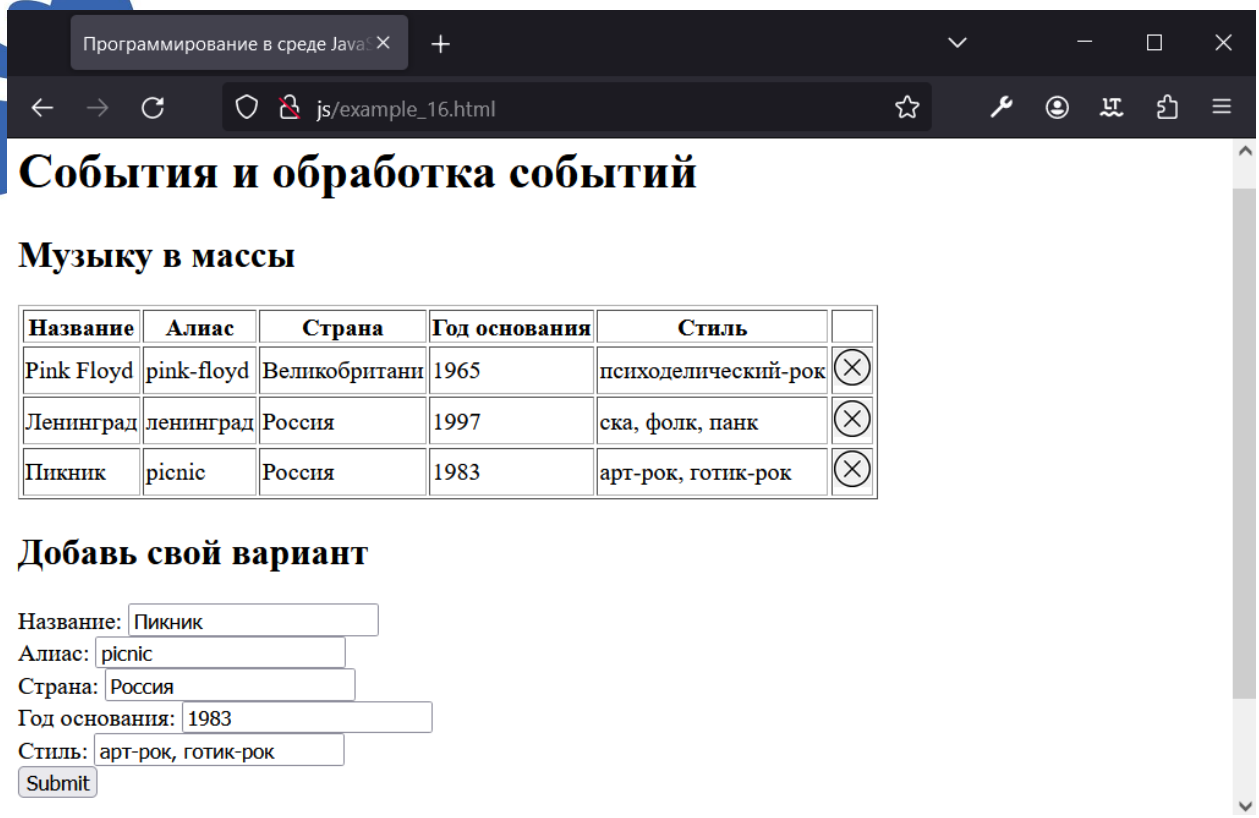
```
<table border="1px">
    <tr>
        <th>Название</th>
        <th>Алиас</th>
        <th>Страна</th>
```



```
            <th>Год основания</th>
            <th>Стиль</th>
            <td></td>
        </tr>
        <tbody id="tbody" onclick="delRow()">
            <tr>
                <td>Aerosmith</td>
                <td>aerosmith</td>
                <td>США</td>
                <td>1970</td>
                <td>хард-рок</td>
                <td><img src='icon.jpg' width="24"></td>
            </tr>
<!-- ... -->
<script>
    // функция добавления строки
    function addRow() {
        // отменим действие по умолчанию (отправка формы)
        event.preventDefault();
        // формирование новой строки
        let row = `
            <tr>
                <td>${frm.elements['name'].value}</td>
                <td>${frm.elements['alias'].value}</td>
                <td>${frm.elements['country'].value}</td>
                <td>${frm.elements['date'].value}</td>
```

```
                <td>${frm.elements['style'].value}</td>
                <td><img src='icon.jpg' width="24"></td>
            </tr>
        `;
        // вставка строки в таблицу
        tbody.insertAdjacentHTML("beforeend", row);
    }
    // функция удаления строки
    function delRow() {
        if (event.target.tagName == "IMG") {
            event.target.closest('tr').style.display = "none";
        };
    }
</script>
```

Формат таблицы с возможность **добавления** / **удаления** строк представлен на следующем рисунке.



В **самостоятельных работах** к уроку выполним следующие задания:

- Создание сценария проверки правильности заполнения полей формы с использованием регулярных выражений.
- Используя делегирование событий создайте скрипт выбора соответствующего рисунка при клике по элементу input.
- Напишите сценарий изменения позиционирования рисунка при клике на ячейке таблицы.

P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).

- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.

К **курсу**:

- **Дополнительный практикум.**